

รายงานสรุปและวิเคราะห์เหตุการณ์ภัยคุกคามทางไซเบอร์

ประเภทภัยคุกคาม: Automated Vulnerability Scanning, File Upload Defect & SQL Injection

สรุปสถานะความรุนแรงในปัจจุบัน: ความเสี่ยงระดับสูงมาก (CRITICAL)

ระบบตรวจพบว่ามีผู้โจมตีสามารถผ่านด่านการตรวจสอบ และนำไฟล์อันตรายเข้าไปบันทึกไว้ในดิสก์ของเครื่องเซิร์ฟเวอร์ (File System) เรียบร้อยแล้ว ควบคู่กับการยิงสคริปต์อัตโนมัติเข้าสู่ฐานข้อมูลผ่านช่องลงทะเบียน/ล็อกอิน จำเป็นต้องดำเนินการจำกัดสิทธิ์และแก้ไขช่องโหว่เพื่อป้องกันการควบคุมระบบโดยสมบูรณ์

1. เกิดอะไรขึ้นกับระบบ? (Root Cause & Attack Analysis)

จากการตรวจสอบหลักฐานทั้งในส่วนของการล็อกไฟล์ที่ถูกอัปโหลดขึ้นเซิร์ฟเวอร์ และ Log ของฐานข้อมูล พบว่าระบบกำลังถูกโจมตีด้วย เครื่องมือสแกนช่องโหว่อัตโนมัติ (Automated Vulnerability Scanner) โดยแฮกเกอร์ไม่ได้ทำตามขั้นตอนแต่ใช้บอทส่งคำสั่งอันตราย (Payloads) เข้ามาทดสอบอย่างต่อเนื่องผ่าน 2 ช่องทางหลัก ดังนี้:

ช่องทางที่ 1: การอัปโหลดไฟล์ (File Upload Manipulation & Arbitrary File Creation)

ผู้โจมตีพบช่องโหว่ในฟังก์ชันการอัปโหลดไฟล์ของเว็บแอปพลิเคชันหรือโมบายแอปพลิเคชัน ส่งผลให้สามารถส่งไฟล์ที่มีโครงสร้างที่ผิดปกติเข้าไปค้างบนระบบได้ โดยแบ่งลักษณะการโจมตีจากชื่อไฟล์เป็น 3 ประเภท:

- Path Traversal / Local File Inclusion (LFI):** การส่งชื่อไฟล์ในลักษณะ `../%2f../%2fpasswd` หรือ `%2fetc%2ffpasswd` เป็นการทดสอบว่าระบบยอมให้อัปโหลดไฟล์ข้ามโพลเดอร์ไปกับไฟล์ระบบปฏิบัติการ หรือใช้หลอกล่ออ่านค่าคอนฟิกูเรชันในส่วนอื่นๆ ได้หรือไม่
- Remote Code Execution (RCE) / Arbitrary Code Execution:** การใช้ชื่อไฟล์ที่มีฟังก์ชันโค้ดฝังอยู่ เช่น `'.print(md5(31337)).'` หรือ `;assert(base64_decode(...))` มุ่งเป้าไปที่เซิร์ฟเวอร์ที่ประมวลผลด้วยสคริปต์ (เช่น PHP) เพื่อทดสอบว่าหากระบบนำชื่อไฟล์นี้ไปเรียกใช้ หรือแสดงผลผ่านคำสั่งที่ประมวลผลโค้ดโดยตรง จะทำให้โค้ดอันตรายรันบนเซิร์ฟเวอร์ทันที
- Null-Byte Injection & Extension Bypass:** การใช้ชื่อไฟล์ลงท้ายด้วยอักขระพิเศษ เช่น `win.ini%00` เป็นการหลอกตัวตัดคำ (Parser) ของภาษาเว็บต่างๆ ให้คิดว่าเป็นไฟล์รูปภาพ แต่เวลาบันทึกลงดิสก์จริงกลับกลายเป็นไฟล์คอนฟิกูเรชัน

ช่องทางที่ 2: การฝังคำสั่งฐานข้อมูล (SQL Injection via Username Input)

จาก Log ฐานข้อมูล มีการโจมตีเข้ามาที่ฟิลด์ `username` อย่างชัดเจน เพื่อตรวจสอบสิทธิ์และการตอบสนองของระบบฐานข้อมูล (ในกรณีนี้ระบบใช้ MSSQL เป็นหลัก):

- Boolean-based Blind SQLi:** การใช้คำสั่งทดสอบตรรกะ เช่น `-1 OR 5*5=25 --` เพื่อตรวจสอบว่าถ้าระบบประมวลผลตรรกะที่เป็นจริง (`25=25`) หน้าเว็บจะแสดงผลต่างจากตรรกะที่เป็นเท็จหรือไม่ หากต่างกัน แฮกเกอร์จะใช้จุดนี้ในการค่อยๆ ดึงข้อมูลออกมาทีละตัวอักษร

- **Time-based Blind SQLi (MSSQL เฉพาะเจาะจง):** การส่งคำสั่ง `waitfor delay '0:0:15' --` เข้ามา เพื่อสั่งให้ฐานข้อมูลหน่วงเวลาการทำงานไป 15 วินาที หากเว็บเซิร์ฟเวอร์ตอบสนองช้าลง 15 วินาทีจริง แฮกเกอร์จะรู้ทันทีว่าช่องอินพุตนี้เชื่อมต่อไปยังคิวรีฐานข้อมูลโดยตรงโดยไม่มีการกรองข้อมูล

2. แฮกเกอร์จะอย่างไรต่อไป? (Attacker's Next Steps)

เนื่องจากผู้โจมตีทราบแล้วว่าสามารถนำไฟล์ขึ้นไปเก็บบนระบบได้ ขั้นตอนต่อไปที่จะเกิดขึ้นตามทฤษฎีการโจมตี (Cyber Attack Lifecycle) มีดังนี้:

ขั้นตอนถัดไป	พฤติกรรมคาดการณ์ของแฮกเกอร์	ผลกระทบต่อธุรกิจ
1. Web Shell Triggering	ค้นหาตำแหน่งพารามิเตอร์ (Absolute Path) ของไฟล์เดออร์ที่เก็บไฟล์ และลองเรียกเข้าถึงผ่านเบราว์เซอร์โดยตรง เพื่อรันโค้ดและยึดหน้าต่างสั่งการสคริปต์	ระบบถูกควบคุมจากระยะไกล
2. Data Theft & Exfiltration	ขยายผลช่องโหว่ SQL Injection หรือการอ่านไฟล์ระบบเพื่อเข้าถึงตารางผู้ใช้ ดึงรหัสผ่านแฮช และขโมยข้อมูลโค้ดรวมถึงรหัสผ่านเชื่อมต่อฐานข้อมูล (Connection String)	ข้อมูลลูกค้ารั่วไหล / PDPA Violations
3. Lateral Movement & Persistence	ฝังไฟล์ Backdoor ถาวรไว้ในไฟล์เดออร์เล็กๆ เพื่อให้สามารถกลับเข้ามาควบคุมระบบได้ตลอดเวลาแม้ว่าจะมีการรีสตาร์ทเครื่อง หรือปิดช่องโหว่หน้าเว็บในภายหลัง	ภัยคุกคามฝังตัวระยะยาว

3. แนวทางแก้ไขและปฏิบัติการรับมือเหตุการณ์ (Remediation Actions)

เพื่อจะจับความเสียหายอย่างเร่งด่วนและปรับปรุงความปลอดภัยของระบบให้สมบูรณ์ครอบคลุมทั้งหน้าเว็บและฐานข้อมูล ให้ดำเนินงานตามขั้นตอนต่อไปนี้:

มาตรการเร่งด่วนที่สุด (Immediate Mitigation)

- กักกันพื้นที่และทำความสะอาด (Isolate & Clean):** ลบไฟล์แปลกปลอมทั้งหมดที่มีขนาด 0 KB และ 1 KB รวมถึงไฟล์ที่มีรายชื่อและนามสกุลผิดปกติออกจากไฟล์เดออร์เก็บไฟล์อัปโหลดในทันที
- ปิดการทำงานของสคริปต์ในไฟล์เดออร์อัปโหลด (Disable Execution Permissions):** ตรวจสอบการตั้งค่าเว็บเซิร์ฟเวอร์ (เช่น บน IIS หรือ Apache) ปรับปรุงสิทธิ์ของไฟล์เดออร์ที่จัดเก็บไฟล์ให้อนุญาตเฉพาะสิทธิ์ **Read** และ **Write** เท่านั้น และทำการ ยกเลิกสิทธิ์ Execute Scripts / Executable เด็ดขาด เพื่อป้องกันไม่ให้สคริปต์อันตรายทำงานได้แม้จะถูกอัปโหลดขึ้นมาสำเร็จ

การแก้ไขในส่วนของการพัฒนาโปรแกรม (Secure Coding Practices)

เนื่องจากระบบมีการใช้ทั้งภาษา PHP, C# และระบบฐานข้อมูล MSSQL จึงต้องแก้ไขข้อผิดพลาดตามหลักปฏิบัติที่ถูกต้อง ดังนี้:

1. ปรับปรุงได้ระบบอัปโหลดไฟล์ (File Upload Hardening)

ข้อห้าม: ห้ามนำชื่อไฟล์เดิมที่ผู้ใช้อัปโหลดมาตั้งเป็นชื่อไฟล์บนเซิร์ฟเวอร์เด็ดขาด

แนวทางที่ถูกต้อง: ให้ใช้ระบบสุ่มชื่อไฟล์ใหม่ด้วย GUID หรือ Timestamp ร่วมกับการตรวจสอบนามสกุลแบบ Whitelist เท่านั้น ดังตัวอย่างโค้ดในภาษา C# และ PHP ด้านล่าง:

```
// ตัวอย่างการตรวจสอบและเปลี่ยนชื่อไฟล์อย่างปลอดภัยใน C# (.NET)
string originalFileName = Path.GetFileName(uploadFile.FileName);
string extension = Path.GetExtension(originalFileName).ToLower();

// กำหนดลิสต์แบบ Whitelist
var allowedExtensions = new List<string> { ".jpg", ".jpeg", ".png", ".pdf" };
if (!allowedExtensions.Contains(extension)) {
    throw new Exception("ไม่อนุญาตให้อัปโหลดไฟล์ประเภทนี้");
}

// เปลี่ยนชื่อไฟล์ใหม่ป้องกันการทำ Malicious Filename และ RCE
string secureFileName = Guid.NewGuid().ToString() + extension;
string uploadPath = Path.Combine(Server.MapPath("~/Uploads/"), secureFileName);
uploadFile.SaveAs(uploadPath);
```

```
<?php
// ตัวอย่างการเปลี่ยนชื่อไฟล์อย่างปลอดภัยใน PHP
$allowed_ext = array('jpg', 'jpeg', 'png', 'pdf');
$filename = $_FILES['uploaded_file']['name'];
$ext = strtolower(pathinfo($filename, PATHINFO_EXTENSION));

if (!in_array($ext, $allowed_ext)) {
    die("ประเภทไฟล์ไม่ถูกต้อง");
}

// ใช้การสร้าง UUID หรือแฮชแทนชื่อไฟล์เดิม
$secure_name = bin2hex(random_bytes(16)) . '.' . $ext;
move_uploaded_file($_FILES['uploaded_file']['tmp_name'], 'uploads/' . $secure_name);
?>
```

2. ปรับปรุงการใช้งานฐานข้อมูล (SQL Injection Prevention ใน MSSQL)

การใช้ Stored Procedure ใน MSSQL ถือเป็นจุดเริ่มต้นที่ดีมากในการป้องกันภัยคุกคาม แต่ต้องระวังข้อจำกัดสำคัญคือ **ห้ามเขียน Dynamic SQL ต่อตรงภายใน Stored Procedure**

```
-- ❌ ตัวอย่างที่อันตรายมาก (แม้จะเป็น Stored Procedure แต่มีการต่อ String ภายใน)
CREATE PROCEDURE sp_CheckUserWrong
    @Username NVARCHAR(50)
AS
```

```

BEGIN
    -- การนำพารามิเตอร์มาต่อเข้ากับข้อความคำสั่งดิบแบบนี้ แฮกเกอร์สามารถฝังคำสั่งพังระบบเข้ามาได้
    DECLARE @SQL NVARCHAR(MAX) = 'SELECT * FROM Users WHERE Username = ' + @Username
    + ' '
    EXEC(@SQL)
END

-- ตัวอย่างที่ปลอดภัยและถูกต้อง (ใช้ Parameterized Query ภายในสโตร์)
CREATE PROCEDURE sp_CheckUserSecure
    @Username NVARCHAR(50)
AS
BEGIN
    -- ส่งค่าผ่านพารามิเตอร์ของระบบโดยตรง ฐานข้อมูลจะมองค่าที่รับมาเป็นเพียง "ข้อความธรรมดา" ไม่ใช่คำสั่งรัน
    SELECT UserID, Username, Role
    FROM Users
    WHERE Username = @Username;
END

```

นอกจากนี้ ในฝั่งของซอร์สโค้ดโปรแกรม (Application Layer) ที่เรียกใช้งาน Stored Procedure จะต้องส่งค่าผ่านทาง Parameters Object เสมอ ห้ามเขียนข้อความควิรี่ต่อกันตรงๆ ก่อนส่งเข้าฐานข้อมูล